

Shortest Process Next

Predicting short interactive CPU bursts

Definition

Shortest Process Next (SPN) is a scheduling idea that runs the runnable process whose next CPU burst is estimated to be the shortest.

1. Connection to Shortest Job First

Basic Idea

Shortest Job First gives the minimum average response time for batch jobs when the job lengths are known. SPN tries to apply the same idea to interactive systems.

Key Difference

For batch systems, the scheduler may know or approximate the whole job length. For interactive systems, a process repeatedly performs short bursts of work between waits, so the scheduler predicts the length of the next burst.

2. Interactive Process Pattern

Typical Behavior

An interactive process usually follows this cycle:

wait for command → execute command → wait for command → execute command

Scheduling View

If each command execution is treated as a separate small job, then running the shortest command execution first can reduce average response time.



3. Main Problem

Unknown Future

The scheduler does not know which runnable process will have the shortest next CPU burst. It must estimate the next burst from previous behavior.

Practical Rule

Keep an estimated next running time for each process. When choosing among runnable processes, run the process with the smallest estimate.

4. Estimating the Next Burst

Weighted Average

Suppose a process has an old estimate E_{old} , and its newest measured CPU burst is T_{new} . A new estimate can be computed as:

$$E_{\text{new}} = aE_{\text{old}} + (1 - a)T_{\text{new}}$$

Symbol	Meaning
E_{old}	Previous estimate for the process's next CPU burst.
T_{new}	Actual CPU time used during the most recent burst.
a	Weight given to the old estimate, where $0 \leq a \leq 1$.
$1 - a$	Weight given to the newest measured burst.

5. Meaning of a

Memory of the Past

The value of a controls how quickly the estimate reacts to recent behavior.

Choice of a	Effect	Result
a close to 0	New measurements matter more.	Estimate changes quickly.
$a = \frac{1}{2}$	Old estimate and new measurement are balanced.	Simple and practical.
a close to 1	Old estimate matters more.	Estimate changes slowly.

6. Aging

Aging

Aging is the technique of predicting the next value in a sequence by combining the newest measured value with the previous estimate.

Why It Helps

Recent CPU bursts influence the estimate strongly, while older bursts gradually lose importance. This lets the scheduler adapt without being too unstable.

7. Successive Estimates When $a = \frac{1}{2}$

Repeated Updates

Let T_0 be the original estimate. Let T_1, T_2 , and T_3 be the next measured bursts. With $a = \frac{1}{2}$, the estimates become:

$$\begin{aligned}E_0 &= T_0 \\E_1 &= \frac{T_0}{2} + \frac{T_1}{2} \\E_2 &= \frac{T_0}{4} + \frac{T_1}{4} + \frac{T_2}{2} \\E_3 &= \frac{T_0}{8} + \frac{T_1}{8} + \frac{T_2}{4} + \frac{T_3}{2}\end{aligned}$$

Observation

After three new measured bursts, the original estimate T_0 has only weight $\frac{1}{8}$. Older information fades away over time.

$$\begin{aligned}E_1 & \quad \boxed{T_0 : 1/2} \quad \boxed{T_1 : 1/2} \\E_2 & \quad \boxed{T_0} \quad \boxed{T_1} \quad \boxed{T_2 : 1/2} \\E_3 & \quad \boxed{T_0} \quad \boxed{T_1} \quad \boxed{T_2} \quad \boxed{T_3 : 1/2}\end{aligned}$$

8. Why $a = \frac{1}{2}$ Is Easy

Simple Implementation

When $a = \frac{1}{2}$, the new estimate is just the average of the old estimate and the newest measurement:

$$E_{\text{new}} = \frac{E_{\text{old}} + T_{\text{new}}}{2}$$

Efficient Computation

On a binary computer, dividing by 2 can be done with a right shift by 1 bit. This makes the update cheap to implement.

```
for each process:
    store estimated_next_burst

when a process finishes a CPU burst:
    measured_burst = actual CPU time used
    estimated_next_burst = (estimated_next_burst + measured_burst) / 2

when the CPU becomes free:
    choose the runnable process with the smallest estimated_next_burst
```

9. Small Example

Updating an Estimate

Suppose the current estimate is 8 ms and the latest measured burst is 4 ms. With $a = \frac{1}{2}$:

$$E_{\text{new}} = \frac{8 + 4}{2} = 6 \text{ ms}$$

The process is now predicted to need 6 ms next time.

Process	Old Estimate	Latest Burst	New Estimate
A	8 ms	4 ms	6 ms
B	5 ms	9 ms	7 ms
C	12 ms	6 ms	9 ms

Scheduling Decision

After these updates, process A has the smallest estimated burst, so SPN would run A first among these runnable processes.

10. Strengths and Weaknesses

Aspect	Explanation
Main benefit	Can improve average response time by favoring short interactive bursts.
Main challenge	The actual next burst is unknown, so the scheduler depends on prediction.
Good behavior	Processes with consistently short bursts receive quick service.
Risk	Long-burst or CPU-bound processes may wait if short-burst processes keep arriving.

11. Big Picture

Main Conclusion

Shortest Process Next brings the Shortest Job First idea into interactive scheduling by estimating each process's next CPU burst. The estimate is updated from past behavior, commonly using aging.

Exam Shortcut

Remember: SPN chooses the process with the smallest predicted next CPU burst. The prediction is often updated using

$$E_{\text{new}} = aE_{\text{old}} + (1 - a)T_{\text{new}}.$$

With $a = \frac{1}{2}$, old information fades by half each update.

12. Quick Review

Key Points

- Shortest Job First is optimal for minimizing average response time in batch systems when job lengths are known.
- Shortest Process Next adapts this idea to interactive processes.
- Each command execution can be treated like a small job.
- The scheduler cannot know the next CPU burst exactly, so it estimates it.
- The estimate is based on previous behavior of the process.
- A common update rule is $E_{\text{new}} = aE_{\text{old}} + (1 - a)T_{\text{new}}$.
- A small a forgets old behavior quickly.
- A large a remembers old behavior longer.
- With $a = \frac{1}{2}$, updating is simple: add the old estimate and new measurement, then divide by 2.
- Aging makes older measurements gradually lose influence.